# RAPIDS

## *The SciDAC Institute for Artificial Intelligence, Computer Science, and Data*

## Agentic Scientific Workflows

### Scientific Achievement

We develop an agentic framework for automating the entire pipeline for executing a scientific workflow provided in a published paper.

### Significance and Impact

Using LLMs as scientific workflow agents, capable of understanding the requirements of the scientific workflow, orchestrating software installation, simulation and data analysis code generation, workflow configuration and execution, improves scientific productivity by relieving scientists from the burdens of these tasks.

### Research Details

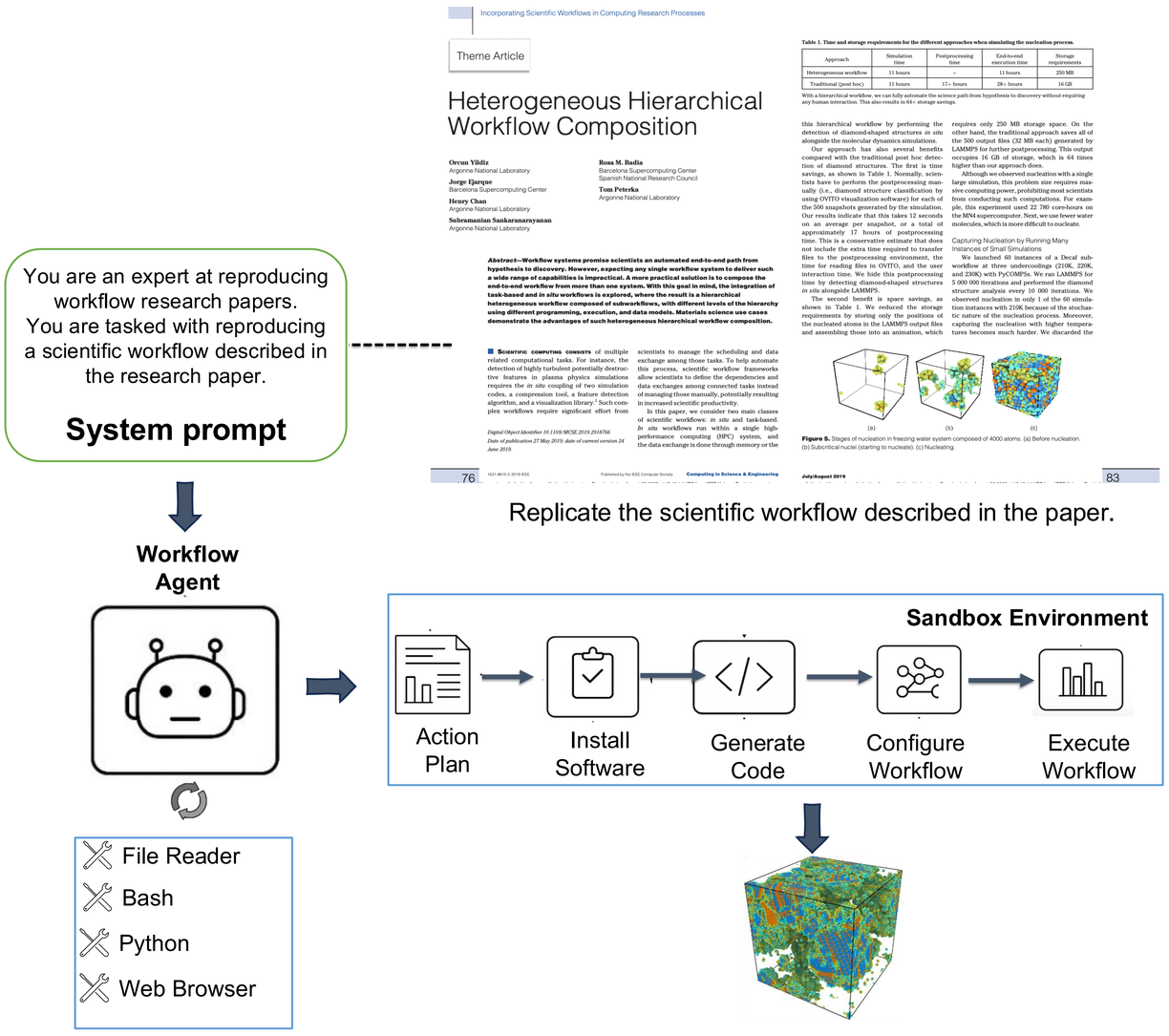
The workflow agent replicates the science use case in the paper by following actions:

- Reads the paper, understands the science use case, and identifies its key tasks
- Creates a plan to replicate the scientific workflow
- Installs the required workflow and task software
- Generates the task codes for simulation and analysis
- Configures and executes the workflow
- Captures outputs and reports results

ASSISTANT
- Reasoning and planned tool usage before actions:
- I first need to read the provided research paper to understand the scientific workflow to reproduce. I used read_file_chunk to read the markdown paper at /home/paper/paper.md in multiple chunks and extract the workflow context, the science use case, and how the tasks compose.
- Then I will implement a Parsl-based workflow using Python to orchestrate two scientific tasks: run a LAMMPS MD simulation (via the Python interface using the provided input files), and post-process the trajectory with Ovito to identify diamond structures and render an image using TachyonRenderer.
- I will use the bash tool to check for or install Python packages (parsl, ovito, lammps) and native libraries needed for Ovito and LAMMPS. I will avoid installing what already exists.
- I will use bash to prepare directories and write configuration and workflow scripts to the filesystem.
- I will execute the workflow with Python and capture logs, then verify the outputs (trajectory frames and the final PNG image).
- I will report results and provide all scripts and steps so the workflow can be reproduced.

***Action plan generated by the workflow agent for replicating the science workflow described in the paper.***

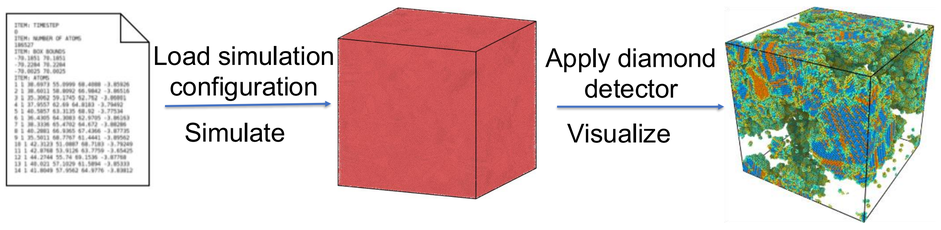
## Workflow Agent




***Overview of the workflow agent framework.***

## Science Use Case

The science use case from materials science is nucleation as a material cools and crystallizes. The workflow agent replicates water freezing workflow, which consists of a molecular dynamics simulation coupled to a diamond structure detector.




- We use the LAMMPS molecular dynamics code for simulating nucleation.
- The diamond structure detector code is available in the OVITO software.
- This detector labels crystalline atoms based on the topological relationship between the nearby first shell and second shell neighbors.
- Unlabeled atoms, including atoms in the grain boundaries, are considered amorphous.

**Simulation code**

```
@python_app
def run_lammps(input_script: str, workdir: str):
    import os
    from lammps import lammps
    # Ensure working directory and frames dir exist
    os.makedirs(os.path.join(workdir, 'frames'),
exist_ok=True)
    # Initialize LAMMPS and run the input script in
the specified workdir
    lmp = lammps()
    cwd = os.getcwd()
    os.chdir(workdir)
    try:
        lmp.file(input_script)
    finally:
        os.chdir(cwd)
        lmp.close()
    return os.path.join(workdir, 'frames')
```

**Analysis code**

```
@python_app
def analyze_with_ovito(frames_glob: str, out_png:
str):
    import os
    from ovito.io import import_file
    from ovito.modifiers import
IdentifyDiamondModifier
    from ovito.vis import TachyonRenderer, Viewport

    # Import trajectory; let Ovito auto-detect
LAMMPS dump across multiple frames
    pipeline = import_file(frames_glob,
multiple_frames = True, sort_particles = True)

    # Add diamond structure identification modifier
    mod = IdentifyDiamondModifier()
    pipeline.modifiers.append(mod)
```

**Visualization code**

```
# Set up a viewport and compute a frame to
initialize scene
    data = pipeline.compute(nframes-1)
    pipeline.add_to_scene()
    vp = Viewport(type = Viewport.Type.Perspective)
    vp.zoom_all()

    # Ensure output directory exists
    os.makedirs(os.path.dirname(out_png),
exist_ok=True)

    # Render with TachyonRenderer
    renderer = TachyonRenderer(shadows=False,
antialiasing=True, antialiasing_samples=2)
    vp.render_image(size=(800,600),
filename=out_png, background=(1,1,1),
renderer=renderer, frame=nframes-1, alpha=False)
    return out_png
```

***Code excerpts from the workflow task codes generated by the workflow agent.***

## ReAct Agent

We use the ReAct agent implemented in the InspectAI framework. The ReAct (Reason + Act) agent has the following capabilities:
- Performs various tool calling functions of LLMs in a loop until the model calls the submit tool to indicate that it has completed the assigned task.
- Adapts to the tool function responses and refines its actions accordingly

**Thought:** I will execute the workflow with Python and capture logs.

**Action:**
```
Bash:
python3 /home/workflow/workflow.py 2>&1 | tee /home/workflow/logs/run.log
```

**Observe:**
```
Traceback (most recent call last):
  File "/home/workflow/workflow.py", line 78, in <module>
    main()
File "/home/workflow/workflow.py", line 25, in run_lammps
    lmp = lammps()
  File "/usr/local/lib/python3.10/dist-packages/lammps/core.py", line 228, in __init__
    self.lib = CDLL(libpath,RTLD_GLOBAL)
  File "/usr/lib/python3.10/ctypes/__init__.py", line 374, in __init__
    self._handle = _dlopen(self._name, mode)
OSError: libmpi.so.12: cannot open shared object file: No such file or directory
```

**Action:**
```
FileReader:
(file: "/usr/local/lib/python3.10/dist-packages/lammps/core.py", start_line: 220, max_lines: 50)
```

**Observe:**
```
…
224:        if name:
225:          libpath = f'liblammps_{name + lib_ext}'
226:        else:
227:          libpath = 'liblammps' + lib_ext
228:      self.lib = CDLL(libpath,RTLD_GLOBAL)
…
```

**Thought:** I need to ensure the environment for LAMMPS shared library.

**Action:**
```
python3 - <<'PY'
import os
from lammps import lammps
os.environ['LD_LIBRARY_PATH'] = '/opt/lammps/src:' + os.environ.get('LD_LIBRARY_PATH','')
print('Attempt to load LAMMPS...')
lmp = lammps()
print('Loaded LAMMPS OK')
PY
```

***ReAct agent tool calling in a loop to install and execute LAMMPS with its Thought-Action-Observe chain.***


O. Yildiz, T. Peterka



Argonne NATIONAL LABORATORY

U.S. DEPARTMENT of ENERGY | Office of Science